CSCE 411: Design and Analysis of Algorithms

Week 14: Randomized Algorithms; Hashing; PageRank

Date: April 21, 2026 *Nate Veldt, updated by Samson Zhou*

1 Randomized Algorithms: Motivating Problems

Randomized algorithms are widely used for processing large datasets and strings. Their power comes from trading exact deterministic behavior for efficiency, while still guaranteeing correctness with high probability.
We now study two representative settings:

- randomized computation models (Monte Carlo vs Las Vegas),
- randomized string processing via hashing (fingerprinting).

1.1 Monte Carlo and Las Vegas Algorithms

Monte Carlo Algorithms. A Monte Carlo algorithm has deterministic runtime but may output _____ with small probability.
Formally, for input x, it outputs $A(x)$ such that

$$\Pr[A(x) \neq f(x)] \leq \delta.$$

Las Vegas Algorithms. A Las Vegas algorithm always outputs the correct answer, but has random runtime.

$$\Pr[A(x) = f(x)] = 1,$$

but the runtime is a _____.

One-sided Error. A Monte Carlo algorithm has one-sided error if it can only fail in one direction (e.g., false positives). _____

Probability Amplification. Repeating an algorithm reduces failure probability to _____ in the number of repetitions, i.e., $\Pr[\text{failure}] \leq e^{-\Omega(k)}$.

Question 1. *Suppose a Monte Carlo algorithm has error probability* $1/3$*. If we run it* k *independent times and take majority vote, what happens?*

A *It remains* $2/3$

B *It approaches* 1 *as* k *increases*

C *It approaches* 0 *as* k *increases*

D *It becomes exactly* 1 *after* $k = 3$

1.2 Fingerprinting and String Matching

We study the string matching problem: given pattern P and text T, find all occurrences of P in T. A naïve algorithm compares each alignment directly, giving runtime _____.

Instead, we use _____, where strings are mapped to compact hash values.
A hash function:

$$h : \Sigma^m \to \mathbb{Z}_M$$

assigns each string a numeric fingerprint.
We compute:

- $h(P)$ for the pattern,
- $h(T[i : i + m])$ for substrings of T.

Key property:

$$P = Q \implies h(P) = h(Q),$$

but equality of hashes does not guarantee equality of strings due to _____.

Rolling Hashes Directly computing each substring hash costs $O(m)$ per position. The goal is to update hashes in _____ as the window shifts.

1.3 The Karp–Rabin Algorithm

We define a rolling hash using base b and modulus M:

$$h(S) = \left(\sum_{i=0}^{m-1} s_i \cdot b^{m-1-i} \right) \bmod M.$$

This treats strings as _____.

Example 1 (numeric alphabet). Suppose the alphabet is digits $\{0, 1, \ldots, 9\}$, and consider the string $S = 314$ with base $b = 10$:

If we take modulus $M = 101$, then:

Example 2 (characters). Map characters to integers, e.g., $a = 1, b = 2, \ldots, z = 26$. Let $S = \texttt{cab}$, with $b = 31$:

Expanding:

If $M = 1000$, then:

Interpretation. The hash is simply evaluating a polynomial at b, then reducing modulo M. Different strings correspond to different polynomials, but distinct polynomials may agree modulo M, leading to _____.

Rolling update rule. When shifting the window:

- remove left character,
- multiply by b,
- add new character.

Thus all substring hashes are computed in total time _____.

Correctness guarantee. If P occurs in T, then their hashes match exactly:

$$h(P) = h(T[i : i + m]).$$

However, we may still have:

$$h(P) = h(T[i : i + m]) \quad \text{but} \quad P \neq T[i : i + m],$$

due to _____.

The probability of collision can be made _____ by choosing large M.

2 Application 1: Checking if Two Strings Match

Fingerprinting gives a randomized method for string equality testing.

Theorem 2.1. *If two strings are identical, then their fingerprints match:*

$$x = y \implies h(x) = h(y).$$

Proof. Equality implies identical polynomial representations, hence identical hashes modulo M.

The converse may fail due to _____:

$$h(x) = h(y) \not\Longrightarrow x = y.$$

Thus fingerprinting is a Monte Carlo equality test: always correct when strings differ, but may fail due to _____.

3 Application 2: Hash Tables and Dictionaries

Given a universe of keys U, a dictionary maintains a dynamic set of elements supporting insert, delete, and search operations.

We resolve collisions using chaining:

1. Map each key to a slot using a hash function $h(k)$.
2. Store all elements that hash to the same slot in a linked list.

Theorem 3.1. *For any deterministic hash function, an adversary can choose n keys that all hash to the same slot, causing search time $\Theta(n)$.*

Proof. Let U be a universe of keys with $|U| > n \cdot m$.

1. When hashing $|U|$ keys into m slots, by the pigeonhole principle there exists a slot containing at least
$$\left\lceil \frac{|U|}{m} \right\rceil > n$$
keys. Reason: _____

2. Since the hash function is deterministic, the adversary can identify a subset of n keys that all map to this same slot.
Reason: _____

3. These keys form a linked list of length n under chaining, so searching requires linear scan.
Reason: _____

Therefore, the worst-case search time is _____. □

4 Universal Hashing and Expected Chain Lengths

If U is a universe of keys and m is the table size, a *universal hash family* $\mathcal{H}$ is a collection of hash functions in which the probability of a collision between any two distinct keys is at most $1/m$.

Let $x, y \in U$ be distinct keys and assume that h is chosen uniformly at random from $\mathcal{H}$. The *expected chain length* for a hash table is bounded as follows:

- There is a random choice of $h \in \mathcal{H}$ at runtime
- There is a collision $h(x) = h(y)$ with probability $\leq 1/m$

Lemma 4.1. *The expected chain length for any key x is* _____

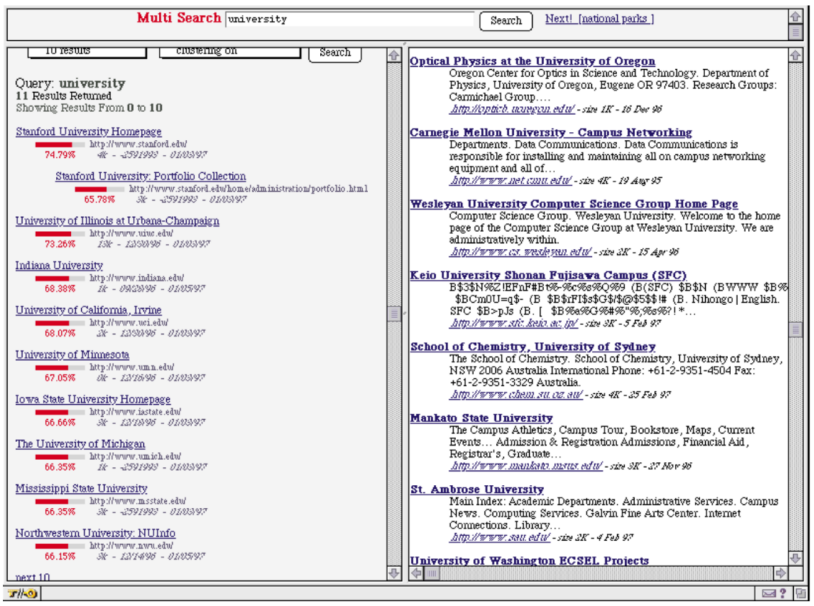
The *load factor* of a hash table with n elements and m slots is α.

$$\alpha = \frac{n}{m}$$

Lemma 4.2. *If we use universal hashing, the expected time for a search operation is*

_____

5 Introduction: Web Search and PageRank



Before Google, ranking web pages was largely based on simple text matching. The **PageRank algorithm**, introduced by Larry Page and Sergei Brin, assigns each webpage a score reflecting its *importance* in the web graph.

Given webpages labeled $1, \ldots, n$, the PageRank of a page i depends on the pages that link to it. However, we do not treat all incoming links equally.

Instead, we weight contributions as follows:

- Links from high-PageRank pages contribute _____.
- Links from pages with many outgoing links contribute _____.

The first principle is circular, but this is exactly what makes the model powerful.

6 BrokenRank (Almost PageRank)

Let $G = (V, E)$ be a directed graph of the web.
Define $L_{ij} = 1$ if $j \to i$, and 0 otherwise. Let $m_j = \sum_{k=1}^{n} L_{kj}$ denote the out-degree of j.

We define a preliminary (but flawed) version of PageRank. The **BrokenRank** p_i is

This reflects our intuition: contributions scale with p_j and are diluted by m_j.

BrokenRank as a Markov Chain

A **Markov chain** is a random process that moves between states $1, \ldots, n$, where each step corresponds to a transition. It is described by an $n \times n$ matrix P, where

Let $p^{(0)}$ be an initial probability distribution. After one step,

Each state collects mass from _____.

Now consider a Markov chain where the states are webpages and the transition matrix is A^T. Since $(A^T)_{ij} = A_{ji} = L_{ji}/m_i$, we have

This corresponds to a *random surfer* who follows outgoing links uniformly at random, with probability _____.

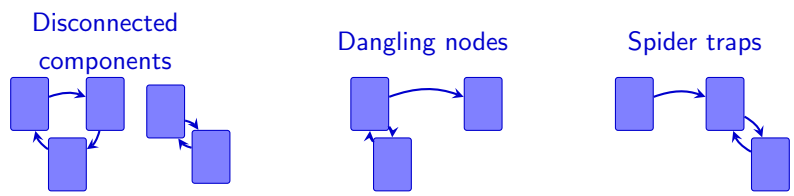
GenericRandomWalk(G)

1. Initialize u to a random page.
2. While the surfer continues:
   (a) Pick an outgoing edge (u, v) uniformly at random.
   (b) Set $u \leftarrow v$.

6.1 Why is BrokenRank Broken?

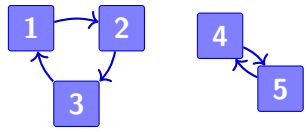
The issue is that this Markov chain is not **strongly connected**. This occurs in several scenarios:

- **Disconnected components.**
- **Dangling nodes (dead ends):** reaching such a node causes probability mass to _____.
- **Spider traps:** a set $S \subseteq V$ with no outgoing edges traps the walk once entered; the surfer _____.



Although a stationary distribution exists, it may be **non-unique**. Thus, BrokenRank is **ill-defined** and depends on the _____.

BrokenRank example



Here $A = LM^{-1} = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}$

Here there are two eigenvectors of A with eigenvalue 1:

$$p = \begin{pmatrix} 1/3 \\ 1/3 \\ 1/3 \\ 0 \\ 0 \end{pmatrix} \qquad \text{and} \qquad p = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1/2 \\ 1/2 \end{pmatrix}$$

These are totally opposite rankings!

If a surfer starts in the first component, pages 1, 2, and 3 receive all the mass; if they start in the second, pages 4 and 5 do. Because of disconnected components, _____.

7 PageRank as a Markov Chain

Key idea. Modify the process so the surfer never gets trapped.

Let E be the $n \times n$ all-ones matrix, i.e., $E_{ij} = 1$ for all i, j. Define

where $d \in (0, 1)$ is the teleportation parameter (typically 0.85).
Then

$$P(\text{go from } i \text{ to } j) = \begin{cases} \frac{1-d}{n} + \frac{d}{m_i} & \text{if } i \to j, \\ \frac{1-d}{n} & \text{otherwise.} \end{cases}$$

This is a **random surfer with teleportation**: with probability d follow a link, otherwise jump uniformly.

Teleportation ensures the walk can always _____, making the chain **strongly connected**. Therefore, the stationary distribution is **unique**.

A *PageRank vector* π satisfies

$$\pi = A^T \pi,$$

i.e., it is a _____.

7.1 Generic Teleportation Update

GenericPageRankUpdate(G, v, d)

1. $jump \leftarrow (1-d)/|V|$
2. $link_sum \leftarrow \sum_{(u,v)\in E} d \cdot \frac{\pi(u)}{d_{\text{out}}(u)}$
3. **return** $jump + link_sum$

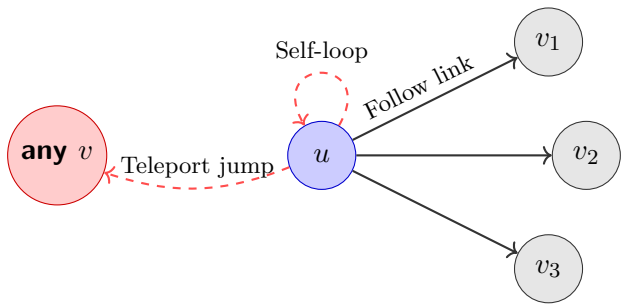
Lemma 7.1. *For any teleportation parameter $d \in (0,1)$, the induced Markov chain is* ***strongly connected****. Consequently, the stationary distribution (the PageRank vector) p is* ***unique****.*

Why does this matter? This lemma is the key guarantee that PageRank is well-defined and computable. In contrast, BrokenRank failed due to structural issues in the web graph: *disconnected components*, *dangling nodes*, and *spider traps*. Teleportation resolves all of these simultaneously.

- **Disconnected components (strong connectivity).** Since $d < 1$, the teleportation probability $(1-d)$ is strictly positive. From any page, there is a nonzero probability of jumping to *any* other page in a single step. Effectively, teleportation introduces _____ between all nodes, making the graph strongly connected.
- **Spider traps and cycles (convergence).** Without teleportation, probability mass can get trapped or cycle indefinitely. Teleportation breaks this behavior by allowing the surfer to leave any set of nodes at any time. In particular, there is always a chance of jumping _____, which acts like a self-loop and prevents periodic oscillations.

7.2 Random Walk Strategies

Strategy	**Transition rule**	**Interpretation**
Simple Random Walk	Follow a uniformly random outgoing edge	Moves only along existing links
Teleportation Walk	Follow a link with prob. d, otherwise jump uniformly	Avoids dead ends and traps



8 Computing PageRank

We compute π via repeated multiplication by A.

PageRank-Power-Iteration(G)

1. Initialize $\pi(u) = 1/|V|$ for all $u \in V$.
2. Choose teleportation parameter d (typically 0.85).
3. Iteratively update scores using incoming edges and teleportation.
4. Output the values once they stabilize.

8.1 Power Iteration Details

We represent the PageRank vector and related quantities as follows:

- $\pi \in \mathbb{R}^V$, where $\pi(u)$ is the PageRank score of node u.
- $d_{\text{out}}(u)$ denotes the out-degree of node u.
- π_{new} is an auxiliary array used for synchronous updates.

Algorithm 1 PageRank-Power-Iteration(G, d)

```
Initialize π(u) ← 1/|V| for all u ∈ V
repeat
    π_new(v) ← (1 − d)/|V|   ∀v ∈ V
    for (u, v) ∈ E do
        π_new(v) ← π_new(v) + d · π(u)/d_out(u)
    end for
    Δ ← max_{v∈V} |π_new(v) − π(v)|
    π ← π_new
until Δ ≤ ε
return π
```

Question 2. *What is the runtime of one iteration of PageRank?*

A $\Theta(\log V)$

B $\Theta(\log E)$

C $\Theta(V + E)$

D $\Theta(E \log V)$

E $\Theta(V^2)$

Given a query:

1. Retrieve the PageRank vector π (precomputed offline).
2. Filter documents containing the query terms.
3. Sort by π and return the top k.